

Session 2

(Shipping to Production)

1- MLflow:-

- It is an Experiment tracking tool for ML or DL problems . it helps you to Store Hyperparameters you used inside your experiment like (ex_name , learning_rate , accuracy , loss , model_name , ...etc) and you can store your dataset used inside the experiment.
- **mlflow dashboard :-**
 - **Run Name :-** the name of the experiment.
 - **Created :-** when you create your experiment.
 - **Duration :-** How long does it take to train that model?
 - **Source :-** source file for your experiment.
 - **Models :-** this is a tag for your champion model it is an alias name to get inside your project.
 - **Hyperparameters :-** your Hyperparameters that are used inside your experiment.
 - **Model_family :-** it is the ML family in your project .
 - you can group by any mlflow parameter (model family , dataset , ...etc)
- **you can compare from selected models.**
- **you can take an overview on your experiment**
 - **who is create it**
 - **source file and source git (branch and commit)**
 - **model tags (model , data set , ...etc)**
 - **hyperparameter**
- **you can upload atrifacts for your experiment (plots , files , photos , ...etx).**

- how to use mlflow inside your code :-
 - set your code on server (localhost:5000)
 - set your experiment name

TOPIC 01 • MLFLOW SETUP



Pointing your code at a tracking server

Two lines before any training code runs

```
train.py
1 # start the server (already in your docker-compose.yml)
2 # docker compose up -d mlflow
3 # then open http://localhost:5000
4
5 import mlflow
6
7 mlflow.set_tracking_uri("http://localhost:5000")
8 mlflow.set_experiment("ride-duration-model")
9
10 # every run from now on lands in that experiment
11 # nothing else in your training code has to change
```

Tracking URI

Where runs are written. Point it at localhost in development and at the team server in CI.

Experiment

A named bucket of runs. One experiment per problem, not per attempt.

Already containerized

MLflow runs as a service beside your API — the same docker-compose file you wrote last session.

06 / 51

- Logging your parameters , matrices and model
- first `mlflow.start_run( )`
- `mlflow.log_params( )` , `mlflow.log_metric( )` , `mlflow.log_model( )`

TOPIC 01 • MLFLOW TRACKING



Logging params, metrics and models

Every training run fully captured, in code

```
train.py
1 import mlflow, mlflow.sklearn
2 from sklearn.ensemble import RandomForestRegressor
3 from sklearn.metrics import mean_absolute_error
4
5 with mlflow.start_run(run_name="rf-baseline") as run:
6
7     params = {"n_estimators": 100, "max_depth": 6}
8     mlflow.log_params(params)
9
10    model = RandomForestRegressor(**params)
11    model.fit(X_train, y_train)
12
13    mae = mean_absolute_error(y_val, model.predict(X_val))
14    mlflow.log_metric("mae", mae)
15
16    mlflow.sklearn.log_model(
17        model, artifact_path="model",
18        registered_model_name="RideDurationModel")
```

start_run()

A context manager — the run closes even if training crashes halfway.

log_params()

A dict of hyperparameters. Searchable and comparable across every run in the UI.

log_metric()

One number tied to this run. Log it repeatedly to get a training curve.

log_model()

Saves weights plus metadata. `registered_model_name` puts it straight into the Registry.

07 / 51

- Model registry



Promote models through stages

None → Staging → Production → Archived

```
registry.py
1 from mlflow import MlflowClient
2
3 client = MlflowClient("http://localhost:5000")
4
5 # list every registered version
6 for v in client.search_model_versions(
7     "name='RideDurationModel'"):
8     print(v.version, v.current_stage)
9
10 # promote version 3 to Staging
11 client.transition_model_version_stage(
12     name="RideDurationModel",
13     version="3", stage="Staging")
14
15 # serving code loads by stage, never by number
16 model = mlflow.sklearn.load_model(
17     "models:/RideDurationModel/Production")
```

None Just registered

Staging Being validated

Production Serving live traffic

Archived Retired

09 / 51

- comparison between (with mlflow ,and without mlflow):-



model_v2_final_REAL.pkl

What experiment tracking replaces

WITHOUT MLFLOW

- model_v1_final.pkl
- model_v1_final_FINAL.pkl
- model_v2_fixed_real.pkl
- Which one scored 0.94 again?
- What hyperparameters did that run use?

WITH MLFLOW

- **Experiment** — a named group of related runs
- **Run** — one training execution with its params and metrics
- **Artifact** — files attached to a run: model, plots, data
- **Registry** — versioned model store with a Staging → Production lifecycle

2- weights & biases :-

- it is like mlflow but have some difference

- difference between mlflow & weights and biases

TOPIC 02 · MLFLOW VS W&B



Same job, different infrastructure

Pick on hosting and team shape, not on features

	MLFLOW	WEIGHTS & BIASES
Hosting	Self-hosted on your server	Cloud SaaS (wandb.ai)
Setup effort	~10 min with docker-compose	pip install + wandb login
Hyperparameter search	Manual, or an Optuna plugin	Built-in sweeps: Bayesian, grid, random
Collaboration	A shared MLflow server	Projects, teams, reports
Artifact storage	Local or S3	W&B cloud artifact store
Cost	Free and open-source	Free tier, paid for teams
Best for	On-prem, enterprise, Kubernetes	Fast iteration, research, reporting

In this course: learn MLflow deeply. Know W&B for research and team reporting.

13 / 51

3- DVC

- It refers to **Data Version Control**, it is a version control to tracking the data.
- it works with **Git**.
- Instead of storing all data versions in one block and every version containing old data + new data , with **DVC** i can do it by uploading every collection of new data as a version and containing a **pointer point to the next collection of data** .
- It saves memory.

- **comparison between (Git without DVC & Git With DVC)**

TOPIC 03 · WHY DVC



Git cannot hold your data

It tracks text. Your dataset is 500MB of it

GIT ALONE

- data/raw/rides.csv — 500 MB
- models/rf_v1.pkl — 200 MB
- models/rf_v2.pkl — 200 MB
- Git refuses files over 100 MB
- History becomes unusable long before that

DVC STORES THE POINTER

- data/raw/rides.csv.dvc — a few hundred bytes
- models/rf_v1.pkl.dvc — a few hundred bytes
- The real files live in S3, GCS or a local cache
- Git tracks the pointers, DVC tracks the data
- Clone the repo, run dvc pull, get the exact dataset

17 / 51

- **you should add your data to the .gitignore file.**
- **you have a config file that has your Data storage locations (local storage , cloud server , ...etc) and you can add any location you want and make it a default .**
- **to add data version to version control**
 - dvc status
 - dvc add file.extension
 - git add file.extension.dvc
 - dvc push
 - git commit -m “your commit text”
- **to change from version to another**
 - git checkout version_name
 - dvc checkout
- **It has a dvc pipeline (dvc.yaml file) . It has stages to track the data and any change, run the pipeline and train , evaluate , and get the model out after change and metrics .**
- **to run the pipeline write (dvc repro)**

- you can use dvc for llm and rag (store knowledge base , chunk store , eval set).

4- Code Quality :-

- It means to make your code clear, readable , organized , compliant with style guidelines , free of common errors ,easy to maintain and suitable for teamwork (all team codes are the same style).
- **Code quality tools :-**
 - **Ruff :-**
 - It is a fast tool to check the code and sometimes solve problems like (unused import , spaces , common problems , issues in code style ,..etc).
 - **you can run it with :-**
 - ruff check .
 - **and fix problems with :-**
 - ruff check . -fix
 - **Black :-**
 - it is an automatic code formatter , format code spaces.
 - **you can run it with :-**
 - black .
 - **Pre-commit :-**
 - It is a script that runs specific tools automatically before the commit (git commit) like Ruff , Black , Tests , if all is good complete the commit .



The config file, in full

Drop it in the repo root and install the hook

```
.pre-commit-config.yaml
1 repos:
2   - repo: https://github.com/astral-sh/ruff-pre-commit
3     rev: v0.4.4
4     hooks:
5       - id: ruff                # linter, auto-fixes safe issues
6         args: [--fix]
7       - id: ruff-format         # formatter, replaces Black
8
9   - repo: https://github.com/pre-commit/pre-commit-hooks
10     rev: v4.6.0
11     hooks:
12       - id: trailing-whitespace
13       - id: end-of-file-fixer
14       - id: check-yaml
15       - id: check-merge-conflict
```

rev is a version pin

Pin the hook version so the whole team formats identically. Bump it deliberately.

args: [--fix]

Ruff repairs what it safely can. What it cannot repair, it blocks the commit on.

The small hooks matter

Trailing whitespace and unresolved merge conflicts cause more review noise than real bugs.

27 / 51

6- Terraform :-

- It is an infrastructure as code (IaC) tool. Instead of creating an infrastructure interface manually from AWS or Azure, write it as a code.
- Terraform code → Terraform apply → AWS , Azure , google cloud , ...etc → Infrastructure Created .
- example of Terraform instructions :-
 - terraform init :-
 - prepare the project and load the provider.
 - terraform plan :-
 - show to you what you will create or edit.
 - terraform apply :-
 - It actually implements the changes.
- why it is important ?
 - create infrastructure quickly.
 - Easily redeploy the same infrastructure.

- control on infrastructure by Git.
- decrease manual errors.
- work with **AWS / Azure / GCP /etc.**

7- CI / CD & GitHub Actions :-

- **CI** refers to **Continuous Integration**. It runs an automatic operation to check that your code is working correctly every time you push the code on Git repo like (check code quality , run ruff , run test ,..etc) if everything is ok , push to repo .
- **CD** refers to **Continuous Deployment**. It deployed the code automatically after **CI** Success Like (build docker image , pushed to docker hub registry , automatic deploy , ..etc).
- **GitHub Actions** helps you to make automation inside GitHub .

TOPIC 06 · WHAT IT IS



Continuous integration and delivery

Automating the verification and packaging of every code change

CI — CONTINUOUS INTEGRATION

- Lint and format check
- Unit and integration tests
- Coverage gate at 80%
- Build verification

CD — CONTINUOUS DELIVERY

- Docker image built
- Image pushed to the registry
- Artifact versioned by git SHA
- Optionally, automatic deploy

Trigger

A push, a PR, a schedule, a manual click

Runner

A clean VM that executes your steps

Steps

Install, test, build, push

Secrets

Encrypted credentials injected at runtime

Artifacts

Coverage reports, images, wheels

8- Integration Testing :-

- it means that to test more than one part of system together (**Like send API Request from frontend to database and return with data**)
- **unit test** test one Function on the system.
- We also have an **End-to-End Test** to test all the system parts together.